

Task 2: Task Details:

1. Create a **custom VPC** with:

Deploy:

- a. One **Public Subnet**
 - b. One **Private Subnet**
 - c. An **Internet Gateway** attached to the VPC
 - d. A **NAT Gateway** in the public subnet
 - e. One **Public EC2 instance** in the public subnet (with Internet Gateway access)
 - f. One **Private EC2 instance** in the private subnet (with outbound access via NAT Gateway)
2. Using the **NAT Gateway**, install **Apache (httpd)** on the **private EC2 instance**.

Solution:

1. Login to your AWS account with credentials and required verification.
(<https://signin.aws.amazon.com/>)
2. Create a custom VPC:
 - Name – *abc*
Assign IPV4 CIDR Block – 10.0.0.0/16
 - (*CIDR block must be in between /16 to /24*)
 - Tendency – Default

VPC > Your VPCs > Create VPC

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional [Info](#)
Creates a tag with a key of 'Name' and a value that you specify.

my-vpc

IPv4 CIDR block [Info](#)
☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR
10.0.0.0/24
CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)
☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy [Info](#)
Default

VPC encryption control (S) [Info](#)
Monitor mode provides visibility into encryption status without blocking traffic. Enforce mode prevents unencrypted traffic. Additional charges apply.

3. Go to subnet and click create subnet.

Public subnet :

- Select the existing VPC you have created.
- Name to subnet (*Public Subnet*)
- Select availability zone
- Give IPV4 CIDR (*10.0.0.0/24*)
- Create it.

The screenshot shows the 'Create subnet' page in the AWS Management Console. The breadcrumb navigation at the top reads 'VPC > Subnets > Create subnet'. The page is divided into two main sections. The top section, 'VPC ID', contains a dropdown menu with the selected value 'vpc-0ecb2db9c6922b0c (my-vpc)'. Below this, the 'Associated VPC CIDRs' section shows 'IPv4 CIDRs' as '10.0.0.0/16'. The bottom section, 'Subnet settings', includes a 'Subnet name' field with the value 'pb-subnet', an 'Availability Zone' dropdown set to 'Europe (Stockholm) / eu-north-1a', and an 'IPv4 VPC CIDR block' dropdown set to '10.0.0.0/16'. The 'IPv4 subnet CIDR block' field is partially visible at the bottom.

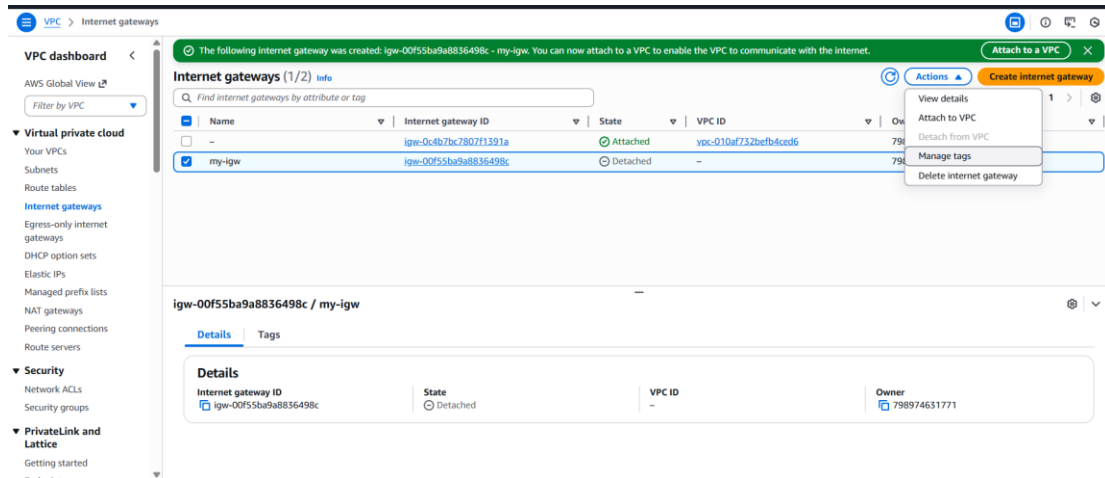
Private subnet :

- Select the existing VPC you have created.
- Name to subnet (*PrivateSubnet*)
- Select availability zone
- Give IPV4 CIDR (*10.0.2.0/24*)
- Create it.

This screenshot shows the 'Create subnet' page for a private subnet. The breadcrumb navigation is 'VPC > Subnets > Create subnet'. The 'VPC ID' dropdown is set to 'vpc-0ecb2db9c6922b0c (my-vpc)'. The 'Associated VPC CIDRs' section shows 'IPv4 CIDRs' as '10.0.0.0/16'. In the 'Subnet settings' section, the 'Subnet name' is 'pr-subnet', the 'Availability Zone' is 'Europe (Stockholm) / eu-north-1a', and the 'IPv4 VPC CIDR block' is '10.0.0.0/16'. The 'IPv4 subnet CIDR block' field at the bottom contains the value '10.0.0.0/20'.

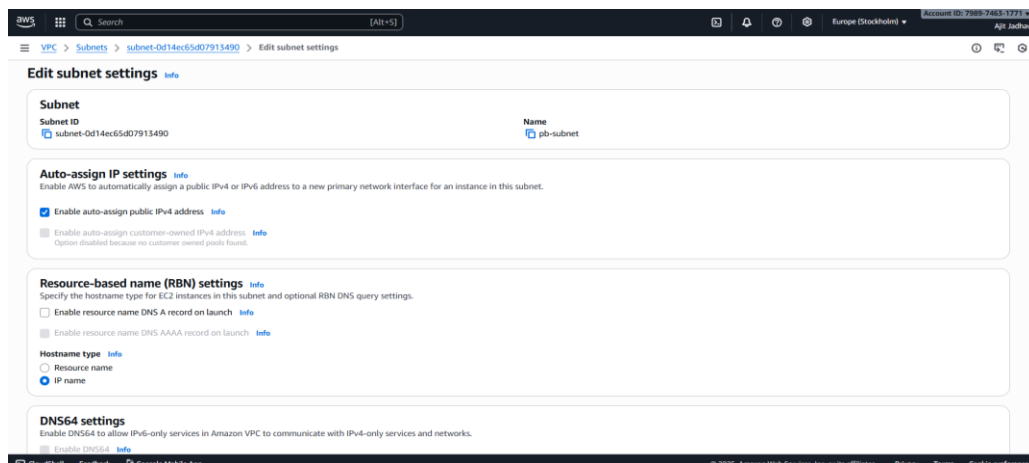
4. Create Internet Gateway to and connect it to VPC.

- Goto Internet Gateway
- Create Gateway
- Give name (*my-igw*)
- Select launched IGW and attach to VPC



5. Create Route table for public subnet

- go to route table
- Create route table
- Use existing VPC
- Go to edit routes add destination and target and save route
- Associate subnet – edit subnet association – select public subnet and save.
- Then Enable Auto assign public Ip- go to subnet – select public subnet – edit subnet setting- then enable option of Auto assign public IP V4 address and save changes.



6. Create NAT gateway for private subnet.

- Create elastic IP and allocate IP
- Goto NAT Gateway – Create NAT gateway- Give name (*my-nat-gw*) - Select availability zone- Select Private subnet- Select Elastic Ip – Create NAT Gateway
- Wait until its available.

The screenshot shows the 'Create NAT gateway' page in the AWS Management Console. The breadcrumb navigation is 'VPC > NAT gateways > Create NAT gateway'. The page contains several sections:

- Name:** A text input field with 'my-nat-gw' entered. A note below states 'The name can be up to 256 characters long.'
- Availability mode:** Two radio buttons are present. 'Regional - new' is selected, with a description: 'Scales automatically across all regional AZs, simplifying management for multi-AZ deployments.' The 'Zonal' option is also visible with a description: 'Provides granular control within a specific availability zone, adhering to subnet level settings.'
- Subnet:** A dropdown menu showing 'subnet-0fbdc774988059531 (pv-subnet)'.
- Connectivity type:** Two radio buttons. 'Public' is selected, with a description: 'Select a connectivity type for the NAT gateway.' The 'Private' option is also visible.
- Elastic IP allocation ID:** A dropdown menu showing 'eipalloc-0bef840961530629a'. An 'Allocate Elastic IP' button is located to the right.
- Additional settings:** A section with an 'Info' link.
- Tags:** A section with a description: 'A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.' It includes a table with columns 'Key' and 'Value - optional'. One tag is listed: 'Name' as the key and 'my-nat-gw' as the value. There are 'Add new tag' and 'Remove' buttons.

7. Create a private route table to private subnet

- Goto route table – create route table – give name to route table (*pvt-routetable*) - allocate existing VPC
- For creating private route for NAT Gateway – Select private route – edit routes- add destination (*0.0.0.0/0*) - select target NAT Gateway and save changes.

The screenshot shows the 'Edit routes' page in the AWS Management Console. The breadcrumb navigation is 'VPC > Route tables > rtb-057b82a3fbfb9ea0c > Edit routes'. The page contains a table with columns: Destination, Target, Status, Propagated, and Route Origin.

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	NAT Gateway	-	No	CreateRoute

Below the table, there is an 'Add route' button. At the bottom right, there are 'Cancel', 'Preview', and 'Save changes' buttons.

- Associate private subnet – go to subnet association – edit subnet association – select private subnet and save changes.

8. Launch Ec2 server in public subnet

- Launch server – give name (*publicserver*) - select OS – select key pair
- Select networking setting- edit- select existing VPC (*my-vpc*) - select public subnet with auto assign IP address.

The screenshot shows the 'Launch an instance' page in the AWS Management Console. The 'Key pair (login)' section has 'testserver' selected. The 'Network settings' section shows 'VPC - required' set to 'vpc-0ecb2db9c6922b0c (my-vpc)' and 'Subnet' set to 'subnet-0d14ec5d07913490' (pb-subnet). 'Auto-assign public IP' is set to 'Enable'. Under 'Firewall (security groups)', 'Create security group' is selected, and the name 'launch-wizard-3' is entered. The 'Summary' panel on the right shows 'Number of instances' as 1, 'Software Image (AMI)' as 'Amazon Linux 2023 AMI', 'Virtual server type (instance type)' as 't3.micro', and 'Storage (volumes)' as '1 volume(s) - 8 GiB'. 'Launch instance' and 'Preview code' buttons are at the bottom right.

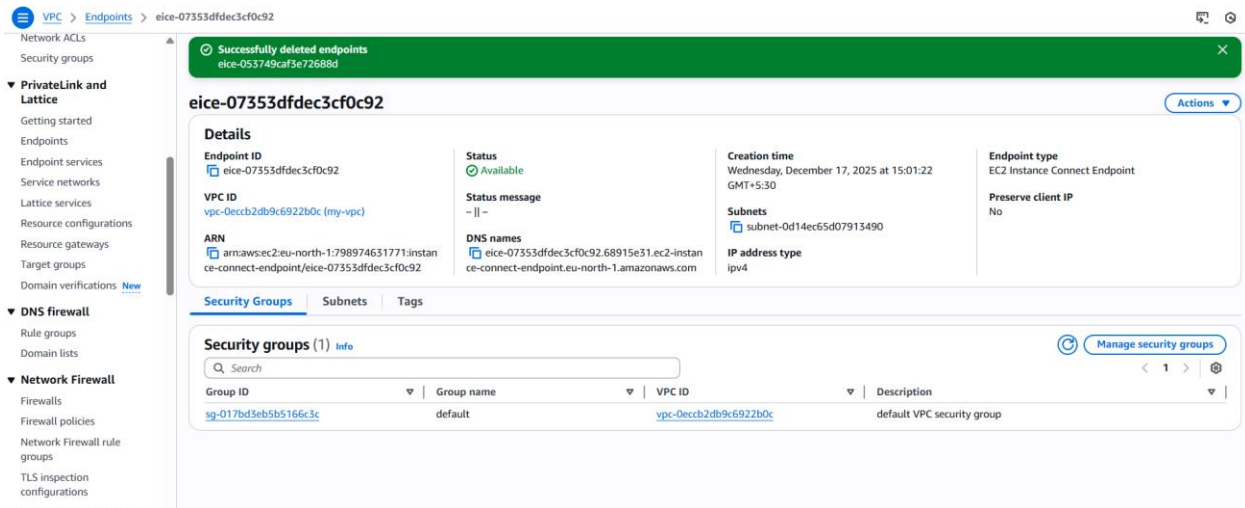
8. Launch Ec2 server in private subnet that internet access only through NAT gateway

- Launch server – give name (*privateserver*) - select OS – select key pair
- Select networking setting- edit- select existing VPC (*my-vpc*) - select private subnet and Disable auto assign IP address.

The screenshot shows the 'Launch an instance' page in the AWS Management Console. The 'Network settings' section shows 'VPC - required' set to 'vpc-0ecb2db9c6922b0c (my-vpc)' and 'Subnet' set to 'subnet-0fbdc774988059531' (pv-subnet). 'Auto-assign public IP' is set to 'Disable'. Under 'Firewall (security groups)', 'Create security group' is selected, and the name 'launch-wizard-3' is entered. The 'Summary' panel on the right is identical to the previous screenshot. At the bottom, the 'Inbound Security Group Rules' section shows a rule for 'ssh' using 'TCP' protocol on port '22'. 'Launch instance' and 'Preview code' buttons are at the bottom right.

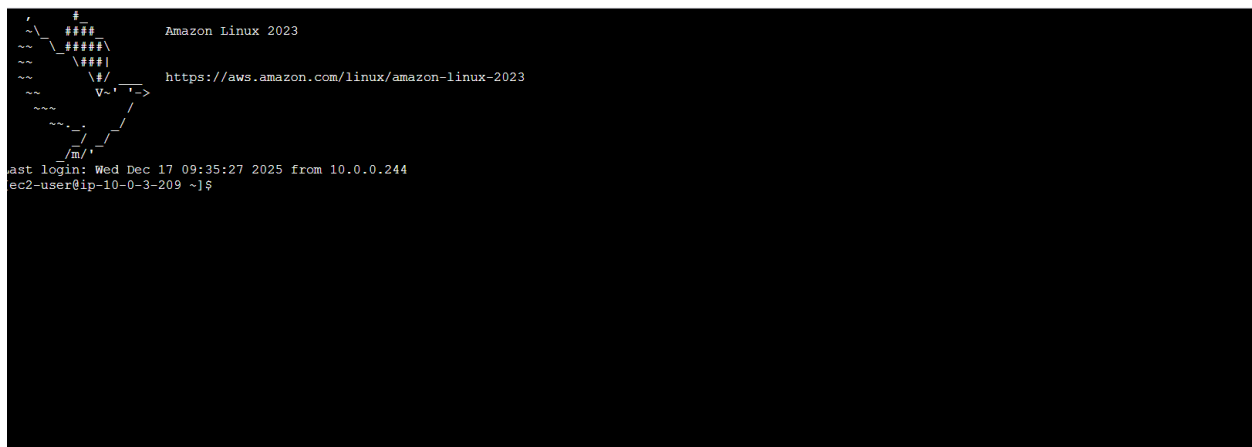
9. For checking Connectivity

- Connect private instance to SSH
- Most important create an Endpoint to access the Ec2 instance by using private IP
- To create endpoint goto VPC – endpoints- Create endpoint – select subnet to endpoint – and VPC
- Add security that we have added to our private instance.
- Create endpoint.



10 . After successfully following such steps, connect to server

Included NAT Gateway for Internet Access for the private Subnet



11. Install Apache on this Server by followed Commands

- `Sudo yum update -y`

- For getting server update
 - *Sudo yum install httpd -y*
 - For installing Apache server
 - *Sudo systemctl start httpd*
 - For used to start Apache
 - *Sudo systemctl enable httpd*
 - For Starting Apache server when logged in to server
 - *Sudo systemctl status httpd*
 - This will show Runing status of Apache server.

```

~# https://aws.amazon.com/linux/amazon-linux-2023
ec2-user@ip-10-0-3-209 ~$ sudo yum install httpd
amazon Linux 2023 Kernel Livepatch repository
No match for argument: httpd
error: Unable to find a match: httpd
ec2-user@ip-10-0-3-209 ~$ sudo yum install httpd
Last metadata expiration check: 0:00:11 ago on Wed Dec 17 09:36:01 2025.
Dependencies resolved.

Package                               Architecture           Version                Repository
-----
Installing:
httpd                                 x86_64                 2.4.65-1.amzn2023.0.2 amazonlinux
Installing dependencies:
apr                                   x86_64                 1.7.5-1.amzn2023.0.4  amazonlinux
apr-util                             x86_64                 1.6.3-1.amzn2023.0.2  amazonlinux
apr-util-ldap                        x86_64                 1.6.3-1.amzn2023.0.2  amazonlinux
generic-logos-httpd                 noarch                 18.0.0-12.amzn2023.0.3 amazonlinux
httpd-core                           x86_64                 2.4.65-1.amzn2023.0.2  amazonlinux
httpd-filesystem                     noarch                 2.4.65-1.amzn2023.0.2  amazonlinux
httpd-tools                          x86_64                 2.4.65-1.amzn2023.0.2  amazonlinux
libbrotli                            x86_64                 1.0.9-4.amzn2023.0.2  amazonlinux
mailcap                              noarch                 2.1.49-3.amzn2023.0.3  amazonlinux
Installing weak dependencies:
apr-util-openssl                     x86_64                 1.6.3-1.amzn2023.0.2  amazonlinux
mod_http2                            x86_64                 2.0.27-1.amzn2023.0.3  amazonlinux
mod_lua                              x86_64                 2.4.65-1.amzn2023.0.2  amazonlinux

```

```

Verifying      : mod_lua-2.4.65-1.amzn2023.0.2.x86_64 13/13

Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64      apr-util-1.6.3-1.amzn2023.0.2.x86_64      apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64      apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch      httpd-2.4.65-1.amzn2023.0.2.x86_64      httpd-core-2.4.65-1.amzn2023.0.2.x86_64      httpd-filesystem-2.4.65-1.amzn2023.0.2.noarch
httpd-tools-2.4.65-1.amzn2023.0.2.x86_64      libbrotli-1.0.9-4.amzn2023.0.2.x86_64      mailcap-2.1.49-3.amzn2023.0.3.noarch      mod_http2-2.0.27-1.amzn2023.0.3.x86_64
mod_lua-2.4.65-1.amzn2023.0.2.x86_64

Complete!
[ec2-user@ip-10-0-3-209 ~]$ sudo systemctl start httpd
Failed to start httpd.service: Unit httpd.service not found.
[ec2-user@ip-10-0-3-209 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-3-209 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-3-209 ~]$ sudo systemctl status httpd
Unit httpd.service could not be found.
[ec2-user@ip-10-0-3-209 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Wed 2025-12-17 09:39:36 UTC; 41s ago
     Docs: man:httd.service(8)
   Main PID: 26465 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
      Tasks: 177 (limit: 1067)
     Memory: 13.2M
        CPU: 91ms
    CGroup: /system.slice/httpd.service
            └─26465 /usr/sbin/httpd -DFOREGROUND
            └─26466 /usr/sbin/httpd -DFOREGROUND
            └─26467 /usr/sbin/httpd -DFOREGROUND
            └─26468 /usr/sbin/httpd -DFOREGROUND
            └─26469 /usr/sbin/httpd -DFOREGROUND

Dec 17 09:39:35 ip-10-0-3-209.eu-north-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Dec 17 09:39:36 ip-10-0-3-209.eu-north-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Dec 17 09:39:36 ip-10-0-3-209.eu-north-1.compute.internal httpd[26465]: Server configured, listening on: port 80
[ec2-user@ip-10-0-3-209 ~]$

```

Successfully Running Apache server on Private Instance